UNIVERSITÉ SAVOIE MONT BLANC • DÉPARTEMENT INFORMATIQUE • UE PROJ531

Quizora — Dossier d'Architecture Technique

Spécifications logicielles, architecture en couches, flux de données et organisation du code

Projet : Application de gestion et passation de quiz en Python (Quizora)
Équipe : Bilal Draoui (Product Owner), Sabrine Kawiche, Chadi Kharchoufa, Gabrien Vaz
Date : Octobre 2026
Version : 2.0 (Sprint 3 - Cœur du jeu)

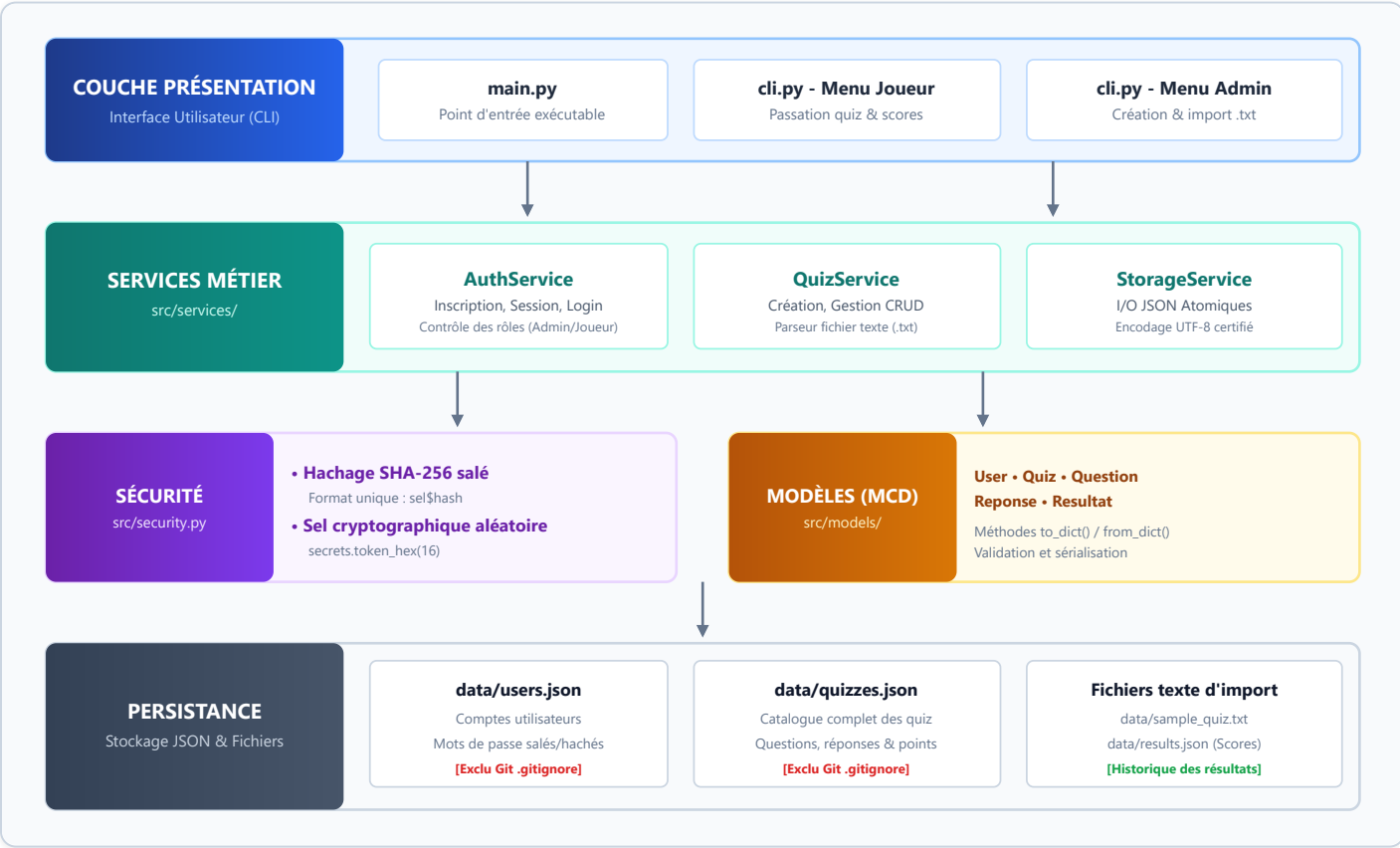
1. Principes Architecturaux & Objectifs de Conception

L'application **Quizora** a été conçue selon le patron d'**architecture en couches (Layered Architecture)**. Cette approche sépare formellement les responsabilités afin de garantir :

- **L'indépendance de l'interface :** L'interface console (CLI) est totalement dissociée des règles métier.
- **La réutilisabilité et la testabilité :** Les services métier et modèles de données peuvent être validés par des tests unitaires automatisés sans nécessiter d'interaction console.
- **La portabilité et la simplicité de déploiement :** Le projet n'exige aucune dépendance externe tierce, s'appuyant rigoureusement sur la bibliothèque standard certifiée de Python.
- **La sécurité native :** Séparation étanche de la gestion des mots de passe salés et hachés, isolée au sein d'un composant cryptographique dédié.

2. Schéma Global de l'Architecture Technique

Le schéma ci-dessous illustre l'organisation des couches logicielles, leurs responsabilités et les flux d'échanges de données :



3. Technologies, Modules & Outils de Gestion de Projet

Le projet s'appuie sur une sélection rigoureuse d'outils professionnels pour assurer l'excellence en gestion de projet (60% de l'évaluation) et en développement technique (40%) :

Domaine / Catégorie	Technologie / Outil	Justification & Rôle dans le projet Quizora
Langage & Runtime	PYTHON 3.10+	Langage principal imposé. Typage dynamique avec annotations d'indices de types (Type Hinting) pour la clarté du code.
Sécurité Cryptographique	HASHLIB & SECRETS	Génération de sel aléatoire de 16 octets (CSPRNG) et hachage SHA-256 avec comparaison à temps constant (`hmac.compare_digest`).
Persistance des Données	JSON (STANDARD LIB)	Sérialisation portable, lisible et standardisée avec encodage forcé en `utf-8` (`ensure_ascii=False`).
Assurance Qualité	UNITTEST	Suite de tests unitaires automatisés validant la non-régression à chaque fin de sprint.
Gestion de Versions	GIT & GITHUB	Dépôt partagé entre les 4 collaborateurs, traçabilité des commits, gestion des branches et fusions.
Gestion de Projet Agile	TRELLO (KANBAN)	Suivi des User Stories (US01 à US07), colonnes *Backlog*, *À faire*, *En cours (WIP)*, *À valider*, *Terminé*.
Planification Temporelle	GANTTPROJECT (.GAN)	Diagramme de Gantt avec affectation des ressources (4 étudiants) et contraintes de durée $\le$ 2 jours.
Modélisation Conceptuelle	LOOPING (.MDC)	Élaboration formelle du Modèle Conceptuel de Données (MCD) selon la méthode Merise.

4. Structure Détaillée de l'Arborescence du Code Source

L'arborescence des fichiers du dépôt est organisée pour séparer la documentation académique, le code applicatif et les tests :

```
PROJ531-Quiz-Python/
│
├── main.py                             # Point d'entrée applicatif unique (lancement : python main.py)
├── README.md                           # Documentation officielle de présentation et d'installation
├── .gitignore                          # Exclusion stricte des données locales et des caches (__pycache__, data/)
│
├── docs/                               # LIVRABLES DE GESTION DE PROJET ET CONCEPTION (60% de la note)
│   ├── Gantt_projet_quiz.gan           # Fichier source du planning (GanttProject)
│   ├── Gantt_projet_quiz.png           # Export visuel haute résolution du diagramme de Gantt
│   ├── wbs_projet_quiz_python.png      # Organigramme de décomposition des tâches (WBS)
│   ├── quiz.mdc                        # Modèle Conceptuel de Données (source Merise Looping)
│   ├── mcd_documentation.md            # Spécifications textuelles et dictionnaire de données MCD
│   ├── glossaire.md                    # Terminologie et glossaire officiel du projet Quizora
│   ├── trello_sprint2.png              # Preuve de validation Kanban Trello du Sprint 2
│   ├── architecture_technique_quizora.pdf  # Le présent document d'architecture logicielle
│   └── comptes-rendus/                 # Comptes-rendus hebdomadaires obligatoires
│       ├── cr_seance_1.md              # Séance 1 : Cadrage, WBS, Gantt, MCD
│       ├── cr_seance_2.md              # Séance 2 : Authentification, sécurité, stockage JSON
│       └── cr_seance_3.md              # Séance 3 : Moteur de quiz, création, import et passation
│
├── src/                                # CODE SOURCE APPLICATIF EN COUCHES
│   ├── cli.py                          # Interface console interactive (CLI) : menus Joueur et Admin
│   ├── security.py                     # Primitives de hachage SHA-256 avec salage cryptographique
│   ├── models/                         # Entités métier pures (sérialisation to_dict / from_dict)
│   │   ├── user.py                     # Modèle Utilisateur (pseudo, email, mdp_hash, role)
│   │   ├── quiz.py                     # Modèle Quiz (titre, description, auteur_id, questions)
│   │   ├── question.py                 # Modèle Question (énoncé, points, liste de réponses)
│   │   ├── reponse.py                  # Modèle Réponse (libellé, indicateur de validité est_correct)
│   │   └── resultat.py                 # Modèle Résultat (score, total, date_passation)
│   │
│   └── services/                       # Logique métier et orchestration
│       ├── auth_service.py             # Authentification, session active et contrôle d'accès
│       ├── quiz_service.py             # Création manuelle, chargement et parseur de quiz texte (.txt)
│       └── storage_service.py          # Gestionnaire de lecture / écriture JSON robuste
│
├── tests/                              # SUITE DE TESTS UNITAIRES AUTOMATISÉS
│   ├── test_security.py                # Validation du salage et de la résistance cryptographique
│   ├── test_auth.py                    # Validation de l'inscription, connexion et rejet des doublons
│   └── test_storage.py                 # Validation de la persistance et intégrité JSON
│
└── data/                               # DONNÉES LOCALES (Non versionnées pour préserver la confidentialité)
    ├── users.json                      # Base de données des utilisateurs enregistrés
    ├── quizzes.json                    # Base de données des quiz et questions
    └── results.json                    # Historique des scores individuels des joueurs
```

5. Flux Fonctionnels et Scénarios d'Interaction

Scénario A : Inscription et Authentification Sécurisée (Sprint 2)

1. L'utilisateur saisit son pseudo, son email et son mot de passe dans `cli.py`.
2. `AuthService` vérifie l'unicité du pseudo et de l'adresse email.
3. `security.py` génère un sel aléatoire de 16 octets et calcule l'empreinte SHA-256 salée : `sel$hash`.
4. L'objet `User` est instancié et transmis à `StorageService` qui l'écrit de manière atomique dans `data/users.json`.

Scénario B : Création Manuelle de Quiz par l'Administrateur (Sprint 3 - US04)

1. L'administrateur accède au menu d'administration sécurisé de Quizora.
2. Saisie du titre et de la description globale du quiz.
3. Boucle interactive d'ajout de questions : énoncé, nombre de points, saisie de 2 à 5 réponses possibles et désignation de la réponse correcte.
4. Création de l'arborescence d'objets (`Quiz -> [Question -> [Reponse]]`) et sauvegarde via `QuizService` dans `data/quizzes.json`.

Scénario C : Importation Automatisée depuis un Fichier Texte (Sprint 3 - US05)

1. L'administrateur renseigne le chemin d'un fichier texte (ex: `sample_quiz.txt`).
2. Le parseur de `QuizService` analyse ligne par ligne les balises (`TITRE:`, `QUESTION:`, `*` pour la bonne réponse, `-` pour les fausses).
3. Validation syntaxique : vérification qu'au moins une réponse correcte est définie par question.
4. Génération automatique des identifiants uniques UUID et intégration dans la base JSON.

Scénario D : Déroulement d'un Quiz & Évaluation par un Joueur (Sprint 3 - US06)

1. Le joueur consulte la liste des quiz disponibles et sélectionne un identifiant de quiz.
2. L'application affiche chaque question séquentiellement avec le numéro des propositions.
3. Le joueur saisit son choix ; le moteur compare la sélection avec les propositions marquées comme correctes.
4. En fin d'évaluation, Quizora restitue immédiatement le score final (ex : `8 / 10 points`), le pourcentage de réussite, et affiche le corrigé détaillé des questions manquées.

6. Matrice de Traçabilité des Exigences du Sujet PROJ531

Exigence du Cahier des Charges	Composant Technique	Statut
Gestion des comptes et utilisateurs autorisés	`AuthService` + `User`	VALIDÉ (SPRINT 2)
Création de quiz directement (manuelle)	`QuizService.creer_quiz()`	EN COURS (SPRINT 3)
Création de quiz à partir d'un fichier texte	`QuizService.importer_fichier_texte()`	EN COURS (SPRINT 3)
Passation de quiz, score final & bonnes réponses	`QuizAppCLI.action_jouer_quiz()`	EN COURS (SPRINT 3)
Historique individuel des scores par joueur	`Resultat` + `results.json`	PLANIFIÉ (SPRINT 4)
Dépôt Open Source sans bug & Tests unitaires	`tests/` (15+ tests unitaires)	100% RÉUSSI

> **Document certifié par l'Équipe Quizora :** Rédigé sous la supervision du Product Owner Bilal Draoui pour la présentation intermédiaire et le dossier technique de l'UE PROJ531.